Pioneer Public Library System

Entity Relationship Diagram | IS 6465 Web Based Applications

Project	Pioneer Public Library System
Artifact	Entity relationship diagram, Phase 1
Prepared by	Cody Schlawin, Cody Adamson, and Jonas Nunn
Version	1.0, October 29, 2026

How to read this diagram

The diagram is read from the outside of a relationship line inward. A label at the end of a line states how many instances of that entity take part: `1..1` means exactly one, `0..1` means none or one, `1..*` means one or more, and `0..*` means none or more. The glyph beside the label is the same statement drawn in Crow's foot: a bar for one, a ring for zero, and a three-prong fork for many, with the bar and the fork combining when a line is both mandatory and multiple.

Inside each entity box the header carries the entity name. Each row below it is one column, and the marker to the left of the name says what kind of column it is. PK marks the primary key and is drawn in red; FK marks a foreign key and UK a unique column, both drawn in green. A row marked PK FK is a subtype table whose primary key is also its foreign key back to Item, which is how the five media types attach to the one title they describe. The Null column in the entity reference in section 3 says whether a column may be left empty.

Every line is drawn at a right angle so that the route it takes through the page can be followed by eye. A line never passes through a box it does not connect, and no two lines share a path, because either would make it impossible to tell which entity a cardinality belongs to.

This is the corrected model. The assignment supplied a suggested ERD and asked for it to be updated; the five changes in section 1 are the substance of that update, and section 5 lists the business rules the corrected model now enforces that the suggested one could not.

1 What changed from the suggested model

Five defects were found in the suggested model. Two are missing entities, one is an attribute placed on the wrong entity, one is a credential table split in two, and two relate to how a key is declared. Each is corrected in the diagram, in the entity reference, and in the relationship reference.

Ref	The suggested model	The problem	The corrected model
a	The model has no Class or ClassSignup entity, though the requirements describe a class catalogue and a signup flow.	UC 16 and FR 15 and FR 16 describe browsing classes, seeing seats remaining, and signing up, and none of that can be stored. A capacity rule cannot be enforced against a table that does not exist.	Class (16) holds the title, schedule, location, instructor, and capacity. ClassSignup (17) holds one row per member per class, with a unique key on the pair so a member cannot hold two places in one class.

Ref	The suggested model	The problem	The corrected model
b	Copy carries Member_ID, so a copy is recorded as belonging to a member.	A physical copy belongs to the collection. It is on the shelf or it is out on loan, and it has no owner among the members. The column also contradicts Checkout, which already records who borrowed which copy, so two tables would disagree about the same fact.	Member_ID is removed from Copy. The borrower is recorded only on Checkout, where the loan dates and both clerks are recorded beside it. Copy keeps its condition and its status.
c	Two credential tables, ACCOUNT for members and ADMIN for staff.	The same login is handled twice, with two password columns, two reset paths, and two places for the role check of BR 10 to be got wrong. Nothing in the requirements distinguishes a member login from a staff login except the role.	One Account table with a nullable Member_ID and a nullable Emp_ID, exactly one of which is set, and an Account_Role of member or admin. One Username namespace, one hash, one place to deactivate a login.
d	DEPARTMENT lists Div_Name as a foreign key to DIVISION.	Div_Name is descriptive text. A name can be edited, and two divisions could share one, so it is neither stable nor unique and cannot identify a parent row.	Div_Code is a CHAR(4) primary key on Division, and Department carries Div_Code as its foreign key. Div_Name stays on Division as plain text and is never referenced.
e	Book, Music, Film, Periodical, and Audiobook carry their own primary keys, Book_ID and so on, alongside ITEM.Item_ID.	The subtype row and the supertype row could not be matched reliably: the item identifier in ITEM pointed at nothing in the subtype, so a title's type and its descriptive fields could drift apart. Nothing stopped two subtype rows claiming to describe one item.	All five subtype tables use Item_ID as both primary key and foreign key into Item. One item row has at most one subtype row, the identifiers are the same value by construction, and Item_Type says which subtype to read.

2 The master diagram

All eighteen entities and all nineteen relationships are on one page. The five columns run from the organisation on the left, through staff and accounts, to circulation, then inventory and money, and finally the media subtypes on the right, so the diagram can be read left to right in the order the library's data is created.

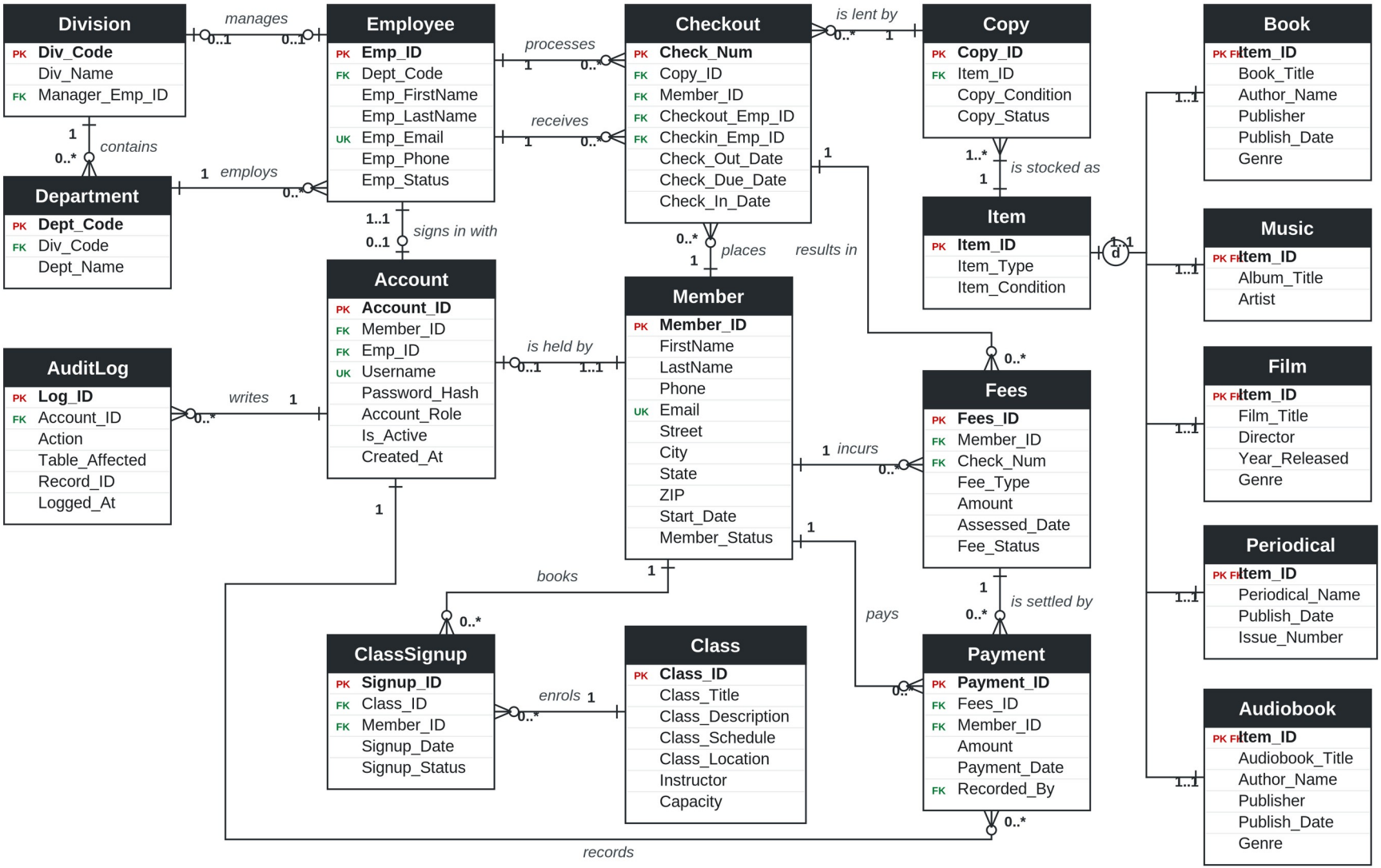


Figure 1. The corrected entity relationship diagram for the Pioneer Public Library System. Eighteen entities, nineteen relationships, Crow's foot cardinality at both ends of every line.

3 Entity reference

Each entity below is listed in the order it appears in the diagram, with the purpose it serves and the columns it carries. Types are the SQL types used in the Phase 1 schema.

Index

#	Entity	Columns	Primary key
1	Member	11	Member_ID
2	Account	8	Account_ID
3	Employee	7	Emp_ID
4	Department	3	Dept_Code
5	Division	3	Div_Code
6	Item	3	Item_ID
7	Book	6	Item_ID
8	Music	3	Item_ID
9	Film	5	Item_ID
10	Periodical	4	Item_ID
11	Audiobook	6	Item_ID
12	Copy	4	Copy_ID
13	Checkout	8	Check_Num
14	Fees	7	Fees_ID
15	Payment	6	Payment_ID
16	Class	7	Class_ID
17	ClassSignup	5	Signup_ID
18	AuditLog	6	Log_ID

1 Member

One person who may borrow copies and sign up for classes. The record survives deactivation.

Column	Type	Key	Null	Notes
Member_ID	INT	PK	no	Surrogate key, assigned on creation, never reused (DR 02).
FirstName	VARCHAR(60)		no	As it appears on the membership form.
LastName	VARCHAR(60)		no	Sort key for the member list and reports.
Phone	VARCHAR(20)		yes	Optional; digits and separators only.
Email	VARCHAR(120)	UK	no	Unique; the login identifier for a member.
Street	VARCHAR(120)		no	Required for an active member.
City	VARCHAR(60)		no	Required for an active member.
State	CHAR(2)		no	Two-letter state code.
ZIP	CHAR(10)		no	Five digits or ZIP+4.
Start_Date	DATE		no	Set to today on insert, never edited (DR 02).
Member_Status	ENUM('active','inactive')		no	Deactivation is a status change, never a delete (BR 06).

2 Account

One credential record for a member or an employee, replacing the suggested model's separate ACCOUNT and ADMIN tables.

Column	Type	Key	Null	Notes
Account_ID	INT	PK	no	One row per set of credentials.
Member_ID	INT	FK	yes	Set for a member login; null for a staff login.
Emp_ID	INT	FK	yes	Set for a staff login; null for a member login.
Username	VARCHAR(40)	UK	no	Unique across both kinds of login.
Password_Hash	VARCHAR(255)		no	One-way hash only; never displayed (FR 33).
Account_Role	ENUM('member','admin')		no	Drives the authorization check in BR 10.
Is_Active	TINYINT(1)		no	0 blocks sign-in without deleting history.
Created_At	DATETIME		no	Written by the server at insert.

3 Employee

A member of staff who may process checkouts and returns or administer the system.

Column	Type	Key	Null	Notes
Emp_ID	INT	PK	no	Staff number, never reused.
Dept_Code	CHAR(4)	FK	no	The department the employee belongs to.
Emp_FirstName	VARCHAR(60)		no	As it appears on the staff record.
Emp_LastName	VARCHAR(60)		no	Used on reports and receipts.
Emp_Email	VARCHAR(120)	UK	no	Unique work address.
Emp_Phone	VARCHAR(20)		yes	Extension or direct line.
Emp_Status	ENUM('active','inactive')		no	Inactive staff keep their checkout history.

4 Department

An organisational unit inside a division.

Column	Type	Key	Null	Notes
Dept_Code	CHAR(4)	PK	no	Short code, for example CIRC.
Div_Code	CHAR(4)	FK	no	The division the department sits in.
Dept_Name	VARCHAR(80)		no	Descriptive only; never used as a key.

5 Division

The top level of the library's organisational structure.

Column	Type	Key	Null	Notes
Div_Code	CHAR(4)	PK	no	Short code, for example PUB.
Div_Name	VARCHAR(80)		no	Descriptive only; never used as a key.
Manager_Emp_ID	INT	FK	yes	At most one manager per division.

6 Item

A title in the collection whatever its media type: the supertype of the five subtype tables.

Column	Type	Key	Null	Notes
Item_ID	INT	PK	no	One row per title, shared by every copy of it.
Item_Type	ENUM('book','music','film','periodical','audiobook')		no	Selects which subtype carries the descriptive fields.
Item_Condition	ENUM('new','good','fair','poor')		no	Default condition applied to each new copy.

7 Book

The descriptive fields that apply only to a book.

Column	Type	Key	Null	Notes
Item_ID	INT	PK FK	no	Same value as Item.Item_ID; no separate Book_ID.
Book_Title	VARCHAR(200)		no	Title as printed.
Author_Name	VARCHAR(120)		no	Primary author.
Publisher	VARCHAR(120)		yes	Imprint name.
Publish_Date	DATE		yes	Date of the edition held.
Genre	VARCHAR(60)		yes	Free text, filtered in reports.

8 Music

The descriptive fields that apply only to a music recording.

Column	Type	Key	Null	Notes
Item_ID	INT	PK FK	no	Same value as Item.Item_ID.
Album_Title	VARCHAR(200)		no	Title of the album.
Artist	VARCHAR(120)		no	Performing artist or group.

9 Film

The descriptive fields that apply only to a film.

Column	Type	Key	Null	Notes
Item_ID	INT	PK FK	no	Same value as Item.Item_ID.
Film_Title	VARCHAR(200)		no	Title as released.
Director	VARCHAR(120)		yes	Primary director.
Year_Released	SMALLINT		yes	Four-digit year.
Genre	VARCHAR(60)		yes	Free text.

10 Periodical

The descriptive fields that apply only to a periodical issue.

Column	Type	Key	Null	Notes
Item_ID	INT	PK FK	no	Same value as Item.Item_ID.
Periodical_Name	VARCHAR(200)		no	Magazine or journal name.
Publish_Date	DATE		yes	Cover date of the issue.
Issue_Number	VARCHAR(30)		yes	Issue or volume label.

11 Audiobook

The descriptive fields that apply only to an audiobook.

Column	Type	Key	Null	Notes
Item_ID	INT	PK FK	no	Same value as Item.Item_ID.
Audiobook_Title	VARCHAR(200)		no	Title as recorded.
Author_Name	VARCHAR(120)		no	Author of the underlying text.
Publisher	VARCHAR(120)		yes	Imprint name.
Publish_Date	DATE		yes	Release date of the edition.
Genre	VARCHAR(60)		yes	Free text.

12 Copy

A single physical copy of a title. It carries no Member_ID; the borrower is recorded on Checkout.

Column	Type	Key	Null	Notes
Copy_ID	INT	PK	no	One row per physical copy on the shelf.
Item_ID	INT	FK	no	The title this copy belongs to.
Copy_Condition	ENUM('new','good','fair','poor')		no	Assessed at check-in.
Copy_Status	ENUM('available','checked out','missing')		no	Exactly one state at a time (BR 08).

13 Checkout

One loan of one copy to one member, open until the copy is returned.

Column	Type	Key	Null	Notes
Check_Num	INT	PK	no	Sequential transaction number.
Copy_ID	INT	FK	no	The copy that left the building.
Member_ID	INT	FK	no	The member who borrowed it.
Checkout_Emp_ID	INT	FK	no	The clerk who processed the loan.
Checkin_Emp_ID	INT	FK	yes	The clerk who took it back; empty while out.
Check_Out_Date	DATE		no	Date the copy left the building.
Check_Due_Date	DATE		no	Check-out date plus 30 days (BR 01).
Check_In_Date	DATE		yes	Empty while the copy is out.

14 Fees

A charge raised against a member, always traceable to the checkout that caused it.

Column	Type	Key	Null	Notes
Fees_ID	INT	PK	no	One row per charge.
Member_ID	INT	FK	no	Who owes the money.
Check_Num	INT	FK	no	The loan that produced the charge.
Fee_Type	ENUM('late','damage')		no	Late or damage.
Amount	DECIMAL(8,2)		no	Money, two decimal places.
Assessed_Date	DATE		no	Date the charge was raised.
Fee_Status	ENUM('unpaid','paid')		no	Unpaid fees feed the BR 03 threshold.

15 Payment

A payment against a fee, with the account that recorded it.

Column	Type	Key	Null	Notes
Payment_ID	INT	PK	no	One row per payment received.
Fees_ID	INT	FK	no	The charge being paid.
Member_ID	INT	FK	no	Who paid.
Amount	DECIMAL(8,2)		no	Amount received.
Payment_Date	DATE		no	Date the payment was taken.
Recorded_By	INT	FK	no	The account that recorded the payment.

16 Class

A class the library offers, with a fixed capacity.

Column	Type	Key	Null	Notes
Class_ID	INT	PK	no	One row per scheduled class.
Class_Title	VARCHAR(120)		no	Shown on the class list.
Class_Description	TEXT		yes	Free text for the class detail page.
Class_Schedule	VARCHAR(120)		no	Readable date and time.
Class_Location	VARCHAR(80)		no	Room or area.
Instructor	VARCHAR(120)		no	Name of the person teaching.
Capacity	SMALLINT		no	Seats available; enforced by BR 13.

17 ClassSignup

A member's place in a class; the unique key on (Class_ID, Member_ID) holds one active signup.

Column	Type	Key	Null	Notes
Signup_ID	INT	PK	no	One row per signup.
Class_ID	INT	FK	no	The class signed up for.
Member_ID	INT	FK	no	The member taking the class.
Signup_Date	DATE		no	Date the signup was made.
Signup_Status	ENUM('confirmed','cancelled')		no	One active signup per member and class (DR 11).

18 AuditLog

One row for every data-changing action, so an action can be traced to an account and a time.

Column	Type	Key	Null	Notes
Log_ID	BIGINT	PK	no	Append-only sequence.
Account_ID	INT	FK	no	The account that performed the action.
Action	VARCHAR(40)		no	create, update, delete, checkout, return, payment.
Table_Affected	VARCHAR(40)		no	The table the action changed.
Record_ID	INT		no	Primary key of the affected row.
Logged_At	DATETIME		no	Server time of the action.

4 Relationship reference

Every relationship in the diagram, with the rule it enforces. The cardinality is written source end first, as it reads on the page.

Relationship	Cardinality	Meaning and rule enforced
Member to Account	1..1 to 0..1	A member has at most one credential record, and a credential record of a member points at exactly one member. Enforced by the nullable Member_ID on Account, unique per member.
Employee to Account	1..1 to 0..1	An employee has at most one credential record. Same Account table as members, distinguished by Account_Role and by which of Member_ID and Emp_ID is set.
Division to Department	1 to 0..*	A division contains any number of departments, and every department sits in exactly one division. Div_Code is mandatory on Department.
Department to Employee	1 to 0..*	A department employs any number of staff, and every employee belongs to exactly one department. Deleting a department is refused while it still has staff.
Employee to Division as manager	0..1 to 0..*	A division has at most one manager, and an employee manages at most one division. Manager_Emp_ID is nullable and unique across Division.
Item to Book, Music, Film, Periodical, Audiobook	1..1 each, disjoint	An item is exactly one of the five media types, never two. The circled d marks the specialisation as disjoint, and each subtype shares Item_ID with its item.

Relationship	Cardinality	Meaning and rule enforced
Item to Copy	1 to 1..*	Every copy is a copy of exactly one title, and a title held by the library has at least one copy. A title with no copies is not stocked and is not created.
Copy to Checkout	1 to 0..*	A copy may be lent many times over its life, one loan at a time. At most one open checkout exists per copy, which is what Copy_Status of checked out records.
Member to Checkout	1 to 0..*	A member may borrow many times, and every loan belongs to exactly one member. A deactivated member keeps their loans.
Employee to Checkout as checkout clerk	1 to 0..*	Every loan records the clerk who processed it. The employee may not be deleted while loans reference them.
Employee to Checkout as check-in clerk	1 to 0..*	The clerk who took the copy back, empty until the copy returns. A separate foreign key from Checkout_Emp_ID so both ends of a loan are attributable.
Checkout to Fees	1 to 0..*	A loan may raise no fee or several, and every fee is traceable to the loan that caused it. A late fee cannot exist without the checkout that ran over.
Member to Fees	1 to 0..*	A member may owe several fees. Fees also reach the member through Checkout; the direct key is kept so a balance can be totalled in one join.
Fees to Payment	1 to 0..*	A charge may be paid in more than one instalment, and every payment settles exactly one charge. Summing payments against a fee cannot exceed the fee.
Member to Payment	1 to 0..*	A member makes many payments. Kept beside Fees_ID so a receipt can name the payer without a join through the charge.
Account to Payment	1 to 0..*	Every payment names the account that recorded it, which is the staff login at the desk. This is the audit trail for money.
Member to ClassSignup	1 to 0..*	A member may sign up for many classes, and each signup belongs to one member.
Class to ClassSignup	1 to 0..*	A class holds many signups up to its Capacity. A signup cannot exist without its class.
Account to AuditLog	1 to 0..*	Every logged action is attributed to exactly one account. The log is append-only, so the account may be deactivated but never deleted.

5 Business rules carried in the model

The schema is the last place a business rule can be enforced, so the rules that the code depends on are visible in the tables rather than only in the application.

Rule	How the model carries it
BR 04 A copy that is checked out cannot be checked out again	Copy_Status is set to checked out when the open Checkout row is created and back to available when Check_In_Date is written. The checkout step reads the status and the open row before it proceeds, so a second loan of the same copy fails on the state rather than on a race.
BR 07 An item cannot be deleted while a copy is out	Copy.Item_ID and Checkout.Copy_ID are declared without cascade delete, so the database refuses the delete while any copy or loan row survives. Deletion is therefore a decision the application has to make deliberately.
BR 06 and DR 14 Nothing is deleted, only deactivated	Member_Status and Emp_Status carry active or inactive, Is_Active on Account blocks sign-in, and Copy_Status carries missing. A member who leaves keeps their loans, fees, and payments, and the history stays reportable.
BR 03 The unpaid-fee threshold	The balance is the sum of Amount over Fees where Fee_Status is unpaid for one member. Fee_Status is the only thing the checkout decision has to read, so a paid fee stops counting the moment it is settled.
BR 08 A copy is in one state at a time	Copy_Status is a single enum of available, checked out, or missing. There is no second flag, so no combination of columns can describe a copy that is both on the shelf and out on loan.
DR 11 One active signup per member per class	Signup_Status carries confirmed or cancelled and the unique key on Class_ID with Member_ID is enforced among confirmed rows, so a member cannot take two places, while a cancelled signup can be made again.

6 Open modeling decisions

- Payment references Fees rather than Checkout. The charge already knows its loan, so the path is Payment to Fees to Checkout. If the library ever takes a payment against a member's whole balance rather than against named charges, Payment will need its own link to Checkout or an allocation table between them.
- Item_Condition sits on Item as the default condition for new copies, while Copy_Condition records the condition of each physical copy. The duplication is deliberate, but if the default is never used it should be dropped from Item so only one place describes condition.
- AuditLog grows without bound and is the one table that never shrinks. Partitioning it by year is the likely next step, and the schema is written so that Logged_At can become the partition key without changing any other table.
- Checkout holds two foreign keys into Employee, one for the clerk who lent the copy and one for the clerk who took it back. This keeps both attributable, but a report that asks how many loans an employee handled has to count both columns, and a loan-processing event table would answer that question more directly.

7 Revision history

Version	Date	Author	Change
1.0	October 29, 2026	Cody Schlawin	First issue. Corrected the five errors in the suggested model and added Class, ClassSignup, Payment, and AuditLog.